

Reproducing and Diagnosing a Hybrid Neural ODE for Plasma Current Dynamics in the Alcator C-Mod Tokamak

Chris Billingham (cmb27@uw.edu), Phillip Prior (pwj32@uw.edu), and Tino Wells (tinow@uw.edu)

Abstract: We reproduce the hybrid neural ODE of Wang, Garnier, and Rea (2023) for plasma dynamics in the Alcator C-Mod tokamak across three independent implementations — Julia/SciML, JAX, and PyTorch — and diagnose how much of the underlying physics the trained models recover. All three reproduce Wang's reported accuracy on the supervised channels, internal inductance L_i and plasma current I_p . A $\dot{V}$ correlation diagnostic shows, however, that the networks do not learn the unsupervised loop-voltage dynamics: predicted dV/dt correlates with finite-differenced data at $R^2 \approx 0.006$ – 0.014 , no better than the analytical Romero baseline, because the finite-differenced V signal is noise-dominated. This forecloses the project's equation-discovery goal, since symbolic regression and SINDy cannot recover structure the data does not contain. The broader caution: a hybrid neural ODE's accuracy on its supervised channels does not establish that it has learned the physics of its unsupervised ones.

Section 1: Introduction and Motivation

1.1 Why fusion, and why now?

Fusion reactors operate by combining the two heaviest isotopes of hydrogen — deuterium (one proton, one neutron) and tritium (one proton, two neutrons) — to produce helium-4 and a high-energy neutron (Figure 1a). The fuel is ionized at temperatures above 100 million Kelvin, becoming a plasma: a state of matter in which electrons separate from nuclei with each charged species responding collectively to electromagnetic fields. Unlike nuclear fission, fusion produces no long-lived high-level radioactive waste and cannot sustain a runaway chain reaction.¹ These properties make fusion energy attractive as a firm, carbon-free baseload — if the engineering can be made to work.

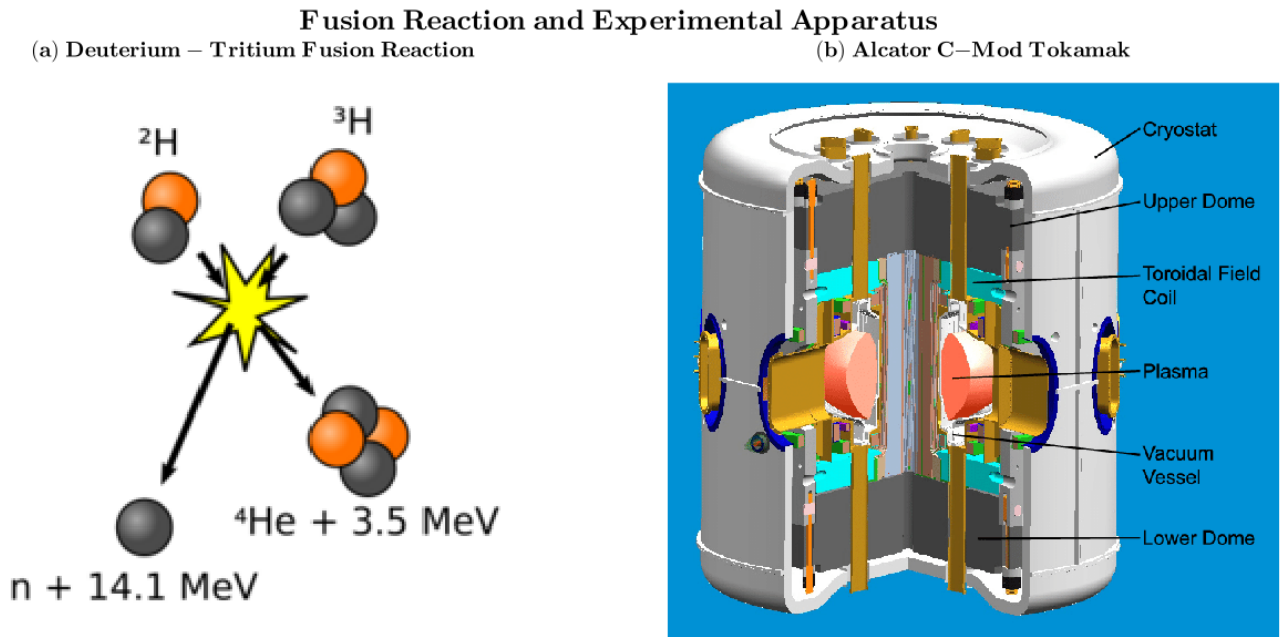


Figure 1: (a) Deuterium-tritium fusion reaction schematic, showing the $D + T \rightarrow \text{He-4} + n$ channel that produces 17.6 MeV per reaction.² (b) Interior of Alcator C-Mod tokamak vessel at MIT (operated 1991–2016), the source of the experimental discharge dataset used in this study.³

For 60 years, progress has been steady but slow toward the central goal of a controlled deuterium-tritium reaction that produces more energy than it consumes.⁴ Fusion is no longer purely a science problem; it is increasingly an engineering and control problem^{5,6} — one whose hardest challenges are being addressed with machine learning.⁷

The engineering bottleneck is plasma stability; a tokamak (Figure 1b) confines a torus-shaped plasma at temperatures above 100 million Kelvin using helical magnetic fields produced by external coils and an induced toroidal current within the plasma itself. The plasma is unstable on millisecond timescales, and a sudden loss of confinement — a "disruption" —

University of Washington ENGR521A June 8, 2026

releases gigajoules of thermal and magnetic energy onto the reactor walls, damaging the hardware and terminating the reaction.⁸ Avoiding disruptions requires real-time control decisions on sub-millisecond timescales that exceed what classical model-based controllers can deliver.

In 2022, Degraeve *et al.* demonstrated that a deep reinforcement learning controller could shape and steer plasma in the TCV tokamak across configurations that lie beyond the reach of conventional control systems.⁹ Neural networks have since been applied to disruption prediction,^{10,11} scenario optimization,¹² and surrogate modeling of plasma transport.¹³ The unifying premise: plasma physics is well-understood in principle but too slow to simulate for real-time control. Data-driven models close that gap, provided they remain physically reasonable.

A purely neural model can fit experimental data without learning the underlying physics, leaving it unreliable when extrapolating to new regimes. Physics-informed machine learning, in which physical structure is built into the model architecture, is one response.^{14,15} A second is to use machine learning to discover governing equations, through symbolic regression^{16,17} or sparse identification of nonlinear dynamics (SINDy).¹⁸ If a trained network can be reduced to an interpretable expression, the result is both fast and physically auditable.

1.2 What we are doing

Our project takes one concrete instance of physics-informed machine learning for plasma dynamics — the hybrid neural ODE proposed by Wang, Garnier, and Rea (2023) for the Alcator C-Mod tokamak (Figure 1b) — and asks three questions.¹⁹ **First**, can we faithfully reproduce Wang's model across three independent implementations — JAX (PP), PyTorch (TW), and Julia (CB)? **Second**, do the trained models learn the underlying physics in a meaningful sense, or do they merely fit the data? **Third**, can interpretable expressions for the dynamics be recovered — either through symbolic regression applied to the trained network, or through sparse identification (SINDy) applied directly to the data?

Wang's model targets three coupled plasma state variables—internal inductance L_i , plasma current I_p , and loop voltage V —driven by an induced voltage V_{ind} from the external poloidal field coils. The dynamics of L_i and I_p are constrained by an analytical physics model from Romero *et al.*,²⁰ and the dynamics of V are learned by a multilayer perceptron. Following Wang, we reproduce both the hybrid (RomeroNNV) architecture and a pure neural ODE baseline (MlpODE) in which all three derivatives come from the network.

1.3 Why this matters

Together, these three questions outline a template for building auditable, real-time-capable physics models for fusion plasmas. A neural ODE that reproduces measured dynamics, passes diagnostic checks, and admits a closed-form symbolic distillation would be substantially more deployable than a black-box network — useful for both real-time control loops and scientific work. Conversely, a negative result on any of these three steps is informative: it tells us where current methods are still gated by the data, the architecture, or the discovery procedure. Either outcome contributes to charting the practical limits of physics-informed machine learning in this domain.

Section 2: Technical Background

This section introduces the four concepts that recur throughout the paper: plasma physics, neural ordinary differential equations, symbolic regression, and sparse identification of nonlinear dynamics.

2.1 Plasma Physics

A tokamak confines plasma in a toroidal magnetic field configuration; controlling the resulting discharge requires tracking three coupled quantities: the plasma current I_p , the internal inductance L_i (a dimensionless measure of how the current is distributed across the plasma cross-section), and the surface loop voltage V driving the current. A fourth quantity, the induced voltage V_{ind} from the external poloidal field coils, enters the dynamics as a known exogenous input rather than a predicted state — it is set by the central solenoid current waveform and read directly from each shot's experimental record. Real-time control over I_p and L_i is essential for safe operation; loss of confinement on millisecond timescales causes plasma disruptions that deposit megajoule-scale energies onto reactor walls. First-principles partial differential equation models of the plasma magnetohydrodynamics are too slow to evaluate during a shot, motivating reduced-order ODE descriptions.

Romero *et al.* derived such a description by enforcing flux and inductive-energy conservation in the plasma column, yielding two exact coupled differential equations for L_i and I_p together with one approximate equation for V :

$$\left. \begin{aligned}
I_p \frac{dL_i}{dt} &= -2V_{ind} - 2V & (1a) \\
L_i \frac{dI_p}{dt} &= 2V_{ind} + V & (1b) \\
\frac{dV}{dt} &\approx -\frac{1}{\tau}V - \frac{\kappa}{\tau}V_{ind} & (1c)
\end{aligned} \right\} \text{Romero} \quad
\left. \begin{aligned}
I_p \frac{dL_i}{dt} &= -2V_{ind} - 2V & (2a) \\
L_i \frac{dI_p}{dt} &= 2V_{ind} + V & (2b) \\
\frac{dV}{dt} &= NN(I_p, L_i, V_{ind}) & (2c)
\end{aligned} \right\} \text{RomeroNN}$$

Equations (1a) and (1b) are exact within the standard tokamak ordering assumptions; they follow directly from Maxwell's equations and the definition of internal inductance. Equation (1c) is fundamentally different: it is a first-order approximation parameterized by a relaxation timescale τ and a coupling coefficient κ . Romero *et al.* reported values of $\kappa = 0.98$ and $\tau = 1.25$ (fit from JET-EFDA experimental discharges), and these were the values used in two of our three reproductions; one implementation instead allowed κ and τ to be trainable parameters, a choice we discuss in Section 3. The loop voltage in a real plasma depends on the spatial structure of the resistivity profile and the current density distribution, neither of which a single-variable ODE can capture analytically. The approximation in (1c) is therefore the natural place for data-driven methods to act — the exact equations (1a) and (1b) are mathematical identities that should be preserved, while (1c) embeds physics that lies beyond what closed-form expressions can describe.

Wang *et al.* exploited this structure to propose two model architectures. The first, which we refer to as RomeroNN (denoted RomeroNNV in ref 19), is a hybrid neural ODE that retains the exact equations (1a) and (1b) and replaces only the approximate V equation (1c) with a neural network. The second, which we call MlpODE, replaces all three governing equations with a single neural network that outputs all three time derivatives directly.

$$\frac{d}{dt}(I_p, L_i, V) = NN(I_p, L_i, V_{ind}) \quad (3a - c) \quad \text{MlpODE}$$

The contrast between (2) and (3) is the central architectural question of the reproduction: does retaining the exact physics for L_i and I_p improve predictive accuracy, and does the neural network in either architecture actually recover the missing V dynamics, or merely fit the observed voltage signal? Sections 4 and 5 examine both questions.

2.2 Neural ODEs

A neural ordinary differential equation (Neural ODE) is a neural network whose output is interpreted as the time derivative of a state vector rather than the state itself.²¹ Given a state $x(t)$, an exogenous control $u(t)$, and a network f_θ with parameters θ , the dynamics are written as

$$\dot{x}(t) = f_\theta(x(t), u(t), t) \quad (4)$$

Trajectories are then generated by integrating these dynamics forward in time using a standard numerical ODE solver. Two properties are particularly important for this work. First, the network is trained to learn state derivatives rather than direct next-step predictions, allowing the learned dynamics to be incorporated naturally within existing physics-based models. Second, the continuous-time formulation allows integration over irregular measurement intervals, which is advantageous for experimental tokamak data that are not sampled at perfectly uniform timesteps.

A *hybrid* neural ODE replaces only the unknown parts of f_θ with a learned network while retaining trusted analytical terms for the remaining state components. In the hybrid RomeroNNV architecture, the analytical Romero equations governing L_i and I_p were retained explicitly, while the voltage-state dynamics $\dot{V}$ were replaced by a neural-network approximation. This structure preserves known plasma physics while allowing the model to learn unmodeled voltage dynamics directly from experimental data.

2.3 Symbolic regression and SINDy

Both methods aim to recover an interpretable mathematical expression for an unknown dynamical law, but they differ in scope. Symbolic regression searches over a combinatorial space of expressions built from primitive operators (+, $\times$, sin, exp, etc.) to find one that fits the data, returning a human-readable equation without prescribing its form. Sparse identification of nonlinear dynamics (SINDy) constrains the search to a *predefined library* of candidate terms and seeks a sparse linear combination that fits, trading expressive generality for speed and explicit sparsity control. In this work, symbolic regression is applied to the *trained network's output* (post-hoc distillation), while SINDy is applied directly to *data residuals* (the part of dV/dt unexplained by Romero physics).

3. Methods: Numerical Implementation

3.1 Wang's Published Model and Our Common Framework

We reproduce the hybrid neural ODE described by Wang *et al.* using both the published manuscript and a partial code snippet shared by the authors in JAX.²² The published methodology is well-specified on several points and underdetermined on others; this section documents both, since reproduction across three frameworks requires explicit choices wherever Wang's specification leaves room.

Dataset. Wang and collaborators selected 489 plasma discharges from the Alcator C-Mod database, filtered for shots with EFIT²³ magnetic-equilibrium reconstructions of internal inductance L_i available and sufficient discharge duration for the modeled dynamics. The dataset is split 70/15/15 into training, validation, and test sets (342 / 73 / 74 shots) using random seed 42; both the split fractions and the seed are specified in the shared code. Each shot contributes 28 timesteps of (L_i, I_p, V, V_{ind}) sampled at a fixed cadence.

Training procedure. Wang trains using multiple shooting, in which each shot is partitioned into segments of length 10 timesteps and the loss is computed across each segment.²⁴ The training loss is a Huber loss²⁵ applied to *relative errors* on L_i and I_p (with outlier threshold $\delta = 0.1$), summed over the segment via the integral-loss wrapper in the shared code. V error is not penalized in training, consistent with Wang's stated methodology of letting the network learn V dynamics implicitly through L_i and I_p fidelity. Optimization uses AdamW with an exponential learning-rate schedule (initial rate 1×10^{-3} , decay factor 0.9 every 50 steps, floor at 2.5×10^{-4}), batch size 512, and early stopping (patience 1,000 epochs of no validation improvement) within a 10,000-epoch ceiling. The neural network is a multilayer perceptron with two hidden layers of 32 units and softplus activations, taking (L_i, I_p, V, V_{ind}) as input and outputting dV/dt for RomeroNN or all three derivatives for MlpODE.

What the published specification leaves underdetermined. Several methodological details could not be fully resolved from the manuscript or the shared code, and our three reproductions had to make explicit choices on them. (i) The shared JAX code applies a normalization constant — referred to as `romero_norm` — in the denominator of the L_i and I_p equations; this constant is provided by `MachineConstants.for_cmod()`, a function not included in the snippet. We could not therefore determine the intended numerical value, and each of our three reproductions made a different choice. (ii) In Wang's pure-physics RomeroModel class — the analytical baseline implementing equation (1c) without a neural network — the parameters κ and τ from Romero's V equation are initialized to Romero's reported values (0.98, 1.25) and the code flags them as trainable, though whether Wang's published baseline results used trained or fixed values cannot be determined from the snippet alone. In Wang's hybrid RomeroNNV class and in the MlpODE architecture, the analytical V equation is replaced by a neural network, so κ and τ do not appear in those models. Our reproductions therefore needed to choose handling of κ and τ only for the pure-Romero baseline and made different choices on this point. (iii) The shared code contained an MlpODE class whose method body was empty (`pass`), indicating Wang's published MlpODE results came from code that was not part of the shared snippet; our three MlpODE implementations are independent reconstructions from the manuscript's architectural description.

We summarize each team's choices on reconstructing these models in the subsections that follow.

3.2 Julia/SciML Implementation

Our Julia implementation matches Wang's setup where the shared code is explicit and makes local choices where Section 3.1 leaves it underdetermined. It uses `Lux.jl`²⁶ for the network, `DifferentialEquations.jl`²⁷ (Tsit5 adaptive solver) with `SciMLSensitivity.jl`²⁸ for adjoint-based gradients, and `Optimization.jl`²⁹ fronting `Optimisers.jl` (AdamW) and `Optim.jl` (L-BFGS). The control V_{ind} is made continuous with a cubic spline (`Dierckx.jl`, $k = 3$) — smooth like Wang's scheme, though Wang used cubic Hermite splines with backward-difference slopes.

Matching Wang: The network is architecturally identical to Wang's: two hidden layers of 32 units, softplus activations, and an unbounded identity output, mapping (L_i, I_p, V, V_{ind}) to dV/dt for RomeroNN or to all three derivatives for MlpODE. The loss is a Huber penalty ($\delta = 0.1$) on the relative errors of L_i and I_p , trapezoidally integrated over each multiple-shooting segment; V is excluded. Multiple shooting uses length-10 segments with one-timestep overlap and zero continuity weight. The 70/15/15 split uses Wang's seed (42), giving the same 342/73/74 shots. Optimization is AdamW under Wang's exponential schedule (1×10^{-3} , decaying by 0.9 every 50 steps, floor 2.5×10^{-4} , weight decay 10^{-4}) for 7,500 iterations.

Diverging from Wang: Four choices differ, each justified locally. (i) *Normalization:* where Wang divides the L_i and I_p equations by `romero_norm = MachineConstants.for_cmod()`, unrecoverable from the snippet, we use a fixed `liip_norm = 0.85`, adopted from the PyTorch default because it best reproduced Wang's accuracy; we could not derive it from Romero 2010 or Wang 2023, and its relation to Wang's machine constant is unconfirmed. (ii) *Initialization:* the final layer is initialized near zero (scaled by 0.01), so RomeroNN begins training as approximately the analytical Romero model and learns a correction from there. (iii) *Gradient clipping:* global-norm clipping at 1.0 (`Optimisers.ClipNorm`), added to stabilize

early training. (iv) *Refinement and selection*: AdamW is followed by up to 500 L-BFGS iterations, and we retain the best-validation-loss checkpoint rather than the final state. The Romero constants $\kappa = 0.98$ and $\tau = 1.25$ enter only the pure-physics baseline; both RomeroNN and MlpODE replace equation (1c) with the network entirely.

The + V variants: Beyond the two Wang-faithful architectures, we trained RomeroNN+ V and MlpODE+ V , which add V to the loss. The V term is a Huber penalty on the variance-normalized residual — rather than relative error, since V passes through zero — weighted $\lambda_V = 0.01$, a pragmatic scale-matching default we did not sweep. Both are trained from scratch with V in the loss. Section 5 reports the results.

Evaluation: On the 74-shot test set we report two Mean Absolute Percentage Error (MAPE) conventions (evaluation only; training stays Huber). Endpoint MAPE takes the error at the final step of each length-10 segment, matching Wang's Table 1; full-shot MAPE averages over all 28 timesteps of each discharge, a stricter test. We also run a V_{dot} correlation diagnostic comparing predicted dV/dt against finite-differenced dV/dt from the data (Section 5).

3.3 Python/JAX Implementation

The JAX implementation uses Equinox for the network and Optax for optimization, adopting the common framework's (§3.1) two-hidden-layer (32-unit, softplus) architecture and relative-error Huber loss ($\delta = 0.1$, trapezoidally integrated), with V excluded from the objective and learned implicitly through its coupling to L_i and I_p . Trajectories are integrated with an explicit fourth-order Runge–Kutta scheme over the measured irregular timesteps, and the control V_{ind} is made continuous by causal cubic-Hermite interpolation with backward-difference slope estimates. Optimization uses AdamW (weight decay 10^{-4}) under an exponential learning-rate schedule (initial 5×10^{-3} , $\times 0.995$ per epoch, floor 1×10^{-4}) with global-norm gradient clipping at 1.0, trained for 3,000 epochs; the minimum-validation-loss checkpoint is retained. The split is 70/15/15 at the shot level.

For RomeroNNV, the analytical Romero equations for L_i and I_p are retained while the network represents V , with two learnable scaling parameters α_{Li} and α_{Ip} introduced into Eq. 2a–2b to absorb the unknown normalization constants in Wang's formulation. For MlpODE, the network represents all three state derivatives directly.

3.4 Python/PyTorch Implementation

The PyTorch implementation introduces a tiered numerical integration strategy and a modified network architecture. Deviating from the manuscript's specified two-hidden-layer softplus setup, this implementation employs a shallower neural network utilizing tanh activations. Trajectory integration is executed using a tiered solver approach: the model initiates integration with an adaptive `Dopri5` solver for its high numerical accuracy. However, to mitigate PyTorch's known susceptibility to numerical underflow—where gradient values fall below the system's floating-point precision, yielding incalculable values—the model dynamically transitions to a fixed-step 4th-Order Runge-Kutta (RK4) solver to maintain stability over the shot trajectories. This stabilization is critically important due to the rigorous evaluation metric employed (addressed below). To resolve the formulation's normalization ambiguity, a static `romero_norm` constant is explicitly applied within the denominators of the exact analytical equations. Optimization is managed via a LambdaLR learning rate schedule across a 3,000-epoch training cycle. Crucially, while the Julia and JAX implementations evaluate using an end-of-segment metric, the PyTorch model computes a full-trajectory trapezoidal-integrated relative error over the entirety of each shot's 28-step continuous trajectory.

4 Results

JAX: The JAX RomeroNNV reproduced Wang's hybrid closely (Table 1: L_i $4.27\% \pm 3.71\%$, I_p $2.83\% \pm 1.52\%$, vs. Wang's $3.50\% / 1.97\%$), with learned scaling parameters $\alpha_{Li} = 0.47$ and $\alpha_{Ip} = 1.91$ absorbing the formulation's normalization uncertainty. The analytical Romero baseline was consistently worse (Table 1), and RomeroNNV converged with little overfitting to roughly an order of magnitude lower loss than MlpODE — retaining the known physics reduces the burden on the learned dynamics.

Three observations concern the V channel. First, averaged over training shots, the measured loop voltage falls through the late discharge while the analytical Romero V stays near-constant (Figure 3), motivating data-driven V models. Second, on a representative test shot (Figure 2) both neural ODEs track L_i and I_p accurately yet infer markedly different V — RomeroNNV rising, MlpODE closer to the measured trend — so V is not uniquely constrained by L_i and I_p . Third, SINDy on the dV/dt residual (full dataset and late-shot subset; Figure 2D–E) explained little of its variance ($R^2 \approx 0.055$), indicating the residual is not captured by a low-order symbolic model.

Julia: The Julia implementation reproduced both architectures in the base configuration (matching Wang's loss, with V excluded) and a $+V$ configuration that adds V to the loss with weight $\lambda_V = 0.01$ — a departure from Wang's methodology. Endpoint MAPE on the 74-shot test set was: RomeroNN $L_i 3.96\% \pm 5.03\%$, $I_p 4.78\% \pm 6.11\%$; MlpODE $L_i 4.02\% \pm 3.44\%$, $I_p 2.67\% \pm 2.59\%$; RomeroNN+V $L_i 4.08\% \pm 4.65\%$, $I_p 5.38\% \pm 7.01\%$; MlpODE+V $L_i 3.00\% \pm 3.47\%$, $I_p 1.44\% \pm 1.47\%$. RomeroNN reproduces Wang's published L_i within 0.5 pp; MlpODE+V achieves

Framework / Source	Model	L_i -MAPE		I_p -MAPE	
		μ_{L_i}	σ_{L_i}	μ_{I_p}	σ_{I_p}
Wang et al.	RomeroNNV	3.50%	3.70%	1.97%	1.90%
	RomeroModel	4.59%	4.37%	2.57%	2.34%
	MlpODE	5.99%	7.23%	8.10%	9.69%
JAX	RomeroNNV	4.27%	3.71%	2.83%	1.52%
	RomeroModel	6.03%	5.36%	9.52%	13.75%
	MlpODE	3.05%	2.58%	2.28%	1.94%
Julia	RomeroNN	3.96%	5.03%	4.78%	6.11%
	RomeroNN+V	4.08%	4.65%	5.38%	7.01%
	RomeroModel	4.83%	5.08%	6.26%	8.42%
	MlpODE	4.02%	3.44%	2.67%	2.59%
	MlpODE+V	3.00%	3.47%	1.44%	1.47%
PyTorch	RomeroNNV	5.34%	5.69%	3.44%	5.88%
	RomeroModel	4.31%	2.41%	2.77%	5.74%
	MlpODE	5.08%	2.21%	0.21%	5.41%

Table 1: MAPE for L_i and I_p across the three frameworks and model variants, compared with Wang et al.'s reference values. Both the metric and the evaluation set differ by framework: JAX, Julia, and the Wang reference values use end-of-segment MAPE on the 74-shot test set, whereas the PyTorch metrics use full-trajectory MAPE on the validation set. The PyTorch I_p values were computed with a signed denominator and are a metric artifact rather than a valid MAPE; they should not be read as true I_p tracking error or compared against the other I_p columns.

the lowest MAPE on both channels of any configuration tested, including Wang's published RomeroNNV. To test whether the trained networks recover the underlying V physics, NN-predicted dV/dt was compared against finite-differenced dV/dt from the data. R^2 remained near zero across all four configurations (0.006–0.014), against a Pure Romero baseline of $R^2 = 0.0002$ — indicating that the networks fit L_i and I_p without learning V dynamics, and that the FD V signal is noise-dominated. The MlpODE+ V MAPE improvement does not reflect recovered V physics; V acts as a free latent variable in the optimization.

PyTorch: The PyTorch implementation evaluated the architectures using a full-trajectory trapezoidal-integrated relative error over the entirety of each shot's 28-step continuous trajectory, contrasting with the end-of-segment metrics utilized in the other frameworks. Following a 3k-epoch training cycle, full-shot MAPE on the validation set was: RomeroNNV $L_i = 5.34\% \pm 5.69\%$, $I_p = 3.44\% \pm 5.88\%$; RomeroModel $L_i 4.31\% \pm 2.41\%$, $I_p = 2.77\% \pm 5.74\%$; MlpODE $L_i = 5.08\% \pm 2.21\%$, $I_p = 0.21\% \pm 5.41\%$. Note that the PyTorch I_p metrics are affected by a sign-convention artifact arising from the use of a signed denominator during integration (see the Table 1 caption). By subjecting the models to compounded error over time, this full-shot integration provides a strictly harder evaluation task than Wang *et al.*'s segment-based metrics. Under this continuous evaluation, the pure physics RomeroModel demonstrated the lowest mean L_i error, narrowing the expected performance delta between the baseline models and the hybrid RomeroNNV architecture.

5. Discussion and Conclusion

Across three independent frameworks we reproduced Wang's hybrid neural ODE and matched or exceeded its published L_i and I_p accuracy: the JAX RomeroNNV (PP) came closest to Wang's hybrid, particularly on I_p , while MlpODE+V reached the lowest test-set MAPE of any configuration (endpoint L_i $3.00\% \pm 3.47\%$, I_p $1.44\% \pm 1.47\%$) — though as a pure neural ODE trained with V in the loss, it departs from Wang's design rather than reproducing it. The V_{dot} diagnostic was uniformly negative: across all four trained configurations the predicted dV/dt correlated with finite-differenced data at $R^2 = 0.006\text{--}0.014$, no better than the closed-form Romero equation ($R^2 = 0.0002$), identifying the noise-dominated FD V signal — not the models — as the binding constraint.

Implications for equation discovery. The project's stretch goal — distilling an interpretable replacement for Wang's approximate V equation via symbolic regression or SINDy — is foreclosed by this result. SR can only fit the network's V output, which is uncorrelated with the physical signal, and SINDy regresses against finite-differenced derivatives carrying the same noise. This sensitivity is well documented — it motivated Weak SINDy³⁰ and Ensemble-SINDy³¹, and the standard caution that finite differencing amplifies measurement noise³². We do not claim V is unlearnable from C-Mod data by *any* method, only that within the methods this dataset was meant to support, signal quality is the limiting factor; noise-robust differentiation or improved V instrumentation are the natural next steps.

The asymmetric effect of adding V to the loss. Adding V to the loss had opposite effects by architecture: it degraded the physics-anchored RomeroNN+V's endpoint I_p ($4.78\% \rightarrow 5.38\%$) by diverting gradient effort onto an unlearnable channel but *improved* the pure MlpODE+V's I_p ($2.67\% \rightarrow 1.44\%$) — its V correlation staying at the noise floor, so the gain reflects regularization rather than recovered physics.

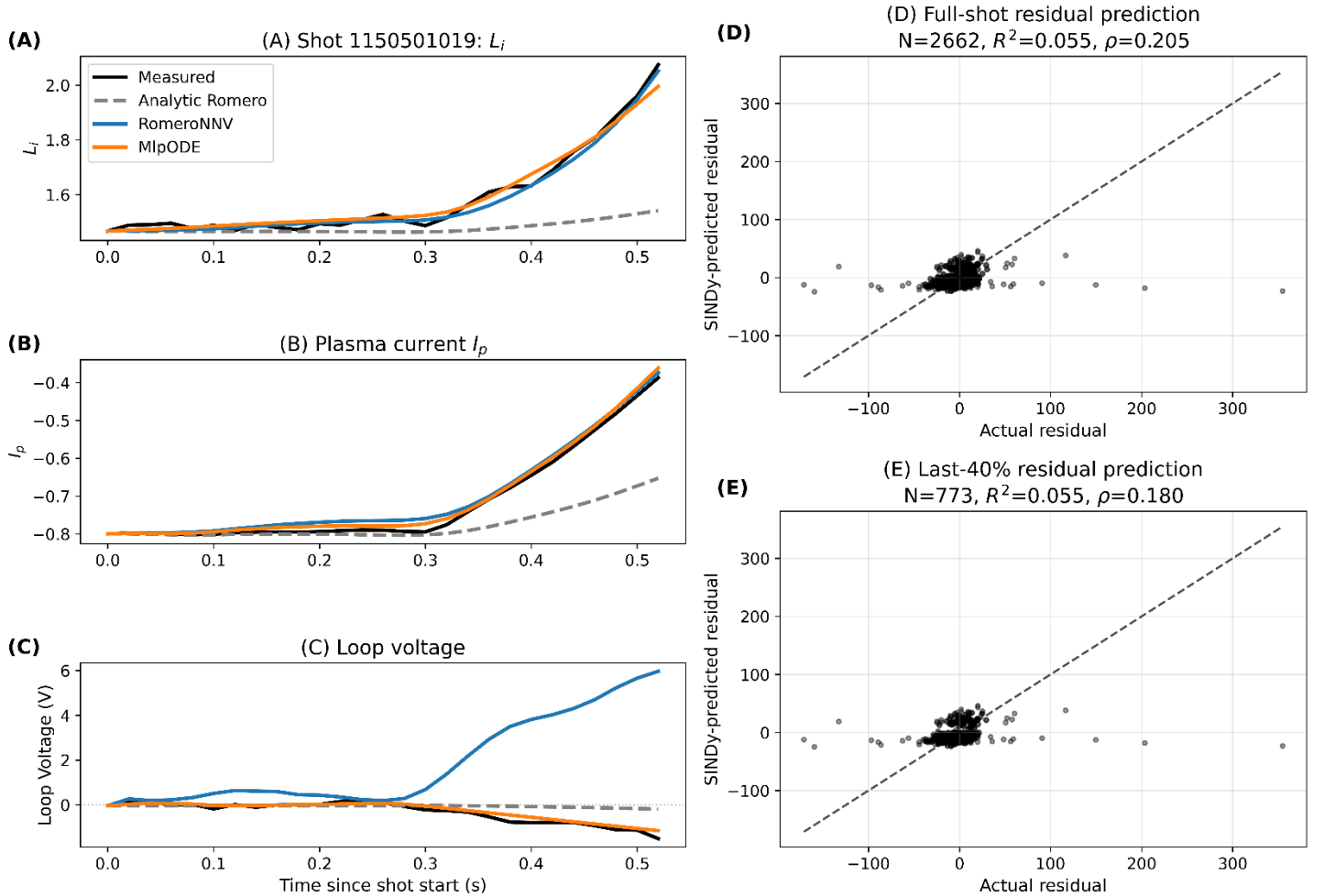


Figure 2: Comparison of analytical and neural-ODE plasma models for representative test shot 1150501019 and subsequent residual analysis. Panels (A)–(C) compare measured trajectories with predictions from the analytical Romero model, the hybrid RomeroNNV model, and the fully neural MlpODE model. Both neural-ODE models accurately reproduce the internal inductance (L_i) and plasma current (I_p), substantially improving upon the analytical Romero formulation. However, the inferred voltage-state trajectories differ markedly, with RomeroNNV predicting a strongly increasing loop voltage while MlpODE remains qualitatively consistent with the measured voltage evolution. Panels (D) and (E) show

SINDy-based residual prediction for the full dataset and the late-shot subset (last 40% of each discharge), respectively. The low coefficients of determination indicate that only a small fraction of the residual variance is explained by the identified symbolic model, suggesting that the remaining discrepancy is not well described by a simple low-order dynamical representation

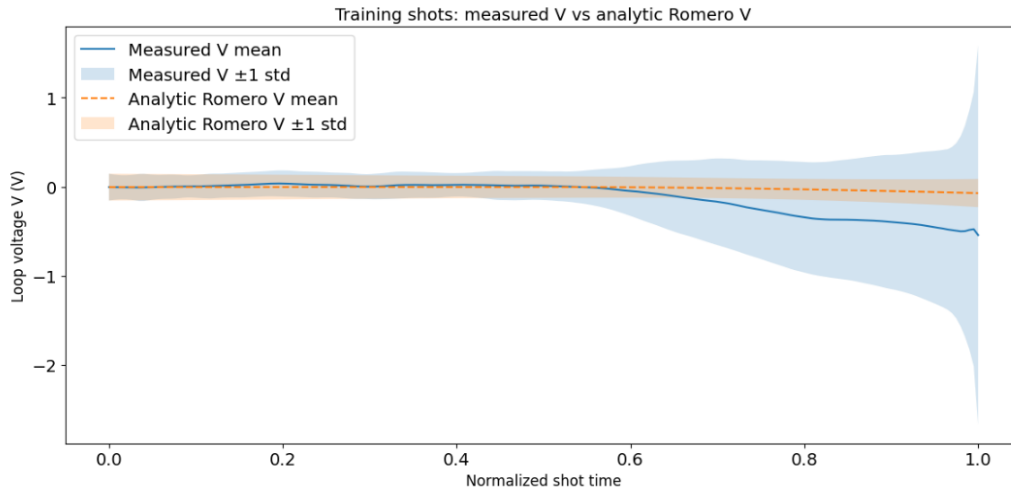


Figure 3: Mean loop-voltage evolution across all training shots compared with the analytical Romero voltage-state model. Shot durations were normalized to the interval $[0,1]$, and ensemble means and standard deviations were computed across the training set. The measured loop voltage exhibits a pronounced negative trend during the final 40% of the discharge, whereas the analytical model predicts voltage trajectories that remain near zero. The growing discrepancy toward the end of the shot suggests that the analytical voltage dynamics do not fully capture the experimentally observed plasma behavior and motivated the development of data-driven voltage-state models.

Julia : Training and Validation Loss Curves — All Four Configurations

(a) Training Loss

(b) Validation Loss

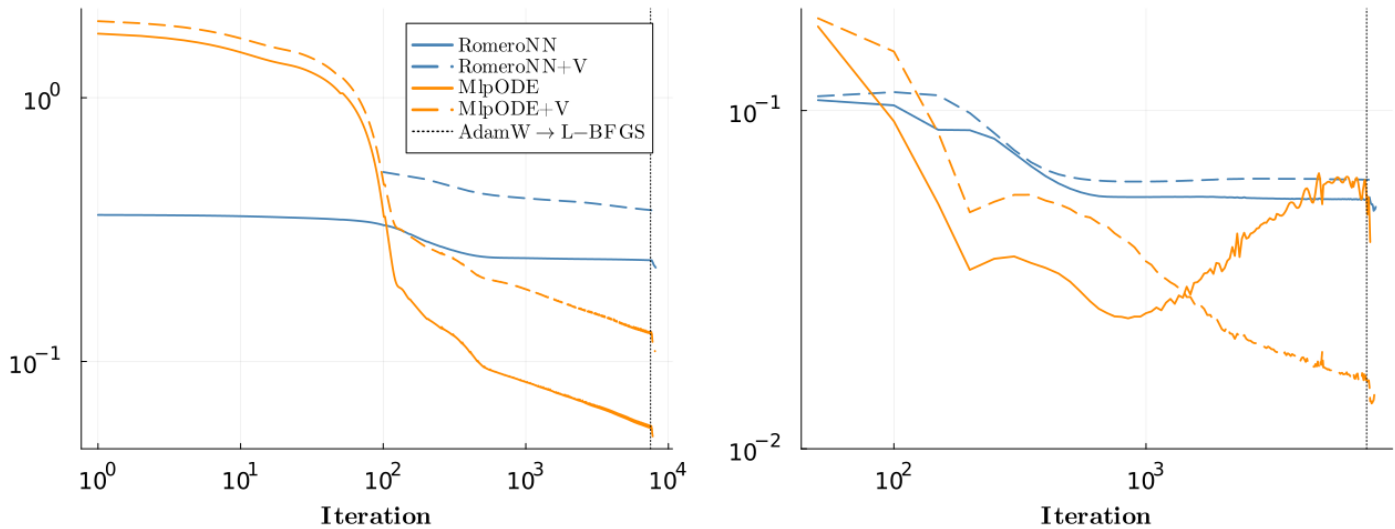


Figure 4: Training (a) and validation (b) Huber loss versus optimizer iteration (log-log) for the four Julia configurations — RomeroNN (blue), MlpODE (orange); base form solid, +V variant dashed. The dotted line at iteration 7,500 marks the AdamW-to-L-BFGS transition. (RomeroNN+V is shown at 50-iteration resolution after a mid-run interruption.)

Reproducibility lessons. Reproducing one paper across three frameworks exposed how underdetermined its published specification was. One gap needed no comparison to spot: the shared snippet's MlpODE class had an empty body, so Wang's published MlpODE results came from code outside the snippet. All three of our MlpODE models are therefore independent reconstructions from the architectural description, and where they outperform Wang's reported MlpODE the gain is not like-for-like — it reflects our own training choices (multiple shooting, L-BFGS refinement, longer training) rather than an identifiable improvement on an implementation we never saw. Other gaps surfaced only because three teams reproduced the paper in parallel: we independently resolved κ and τ (fixed vs. trainable), the normalization constant (and, in JAX,

additional trainable scaling factors), the multiple-shooting overlap differently, and our PyTorch model used a shallower, tanh architecture that departed from the stated network — none of which the paper or snippet flagged as choices. A single reproduction would have settled on one set of these and never registered them as choices at all; that they surfaced through cross-framework comparison is itself a result worth recording.

Julia : Trained Neural ODEs Track L_i and I_p but Not V — shot 1150520003

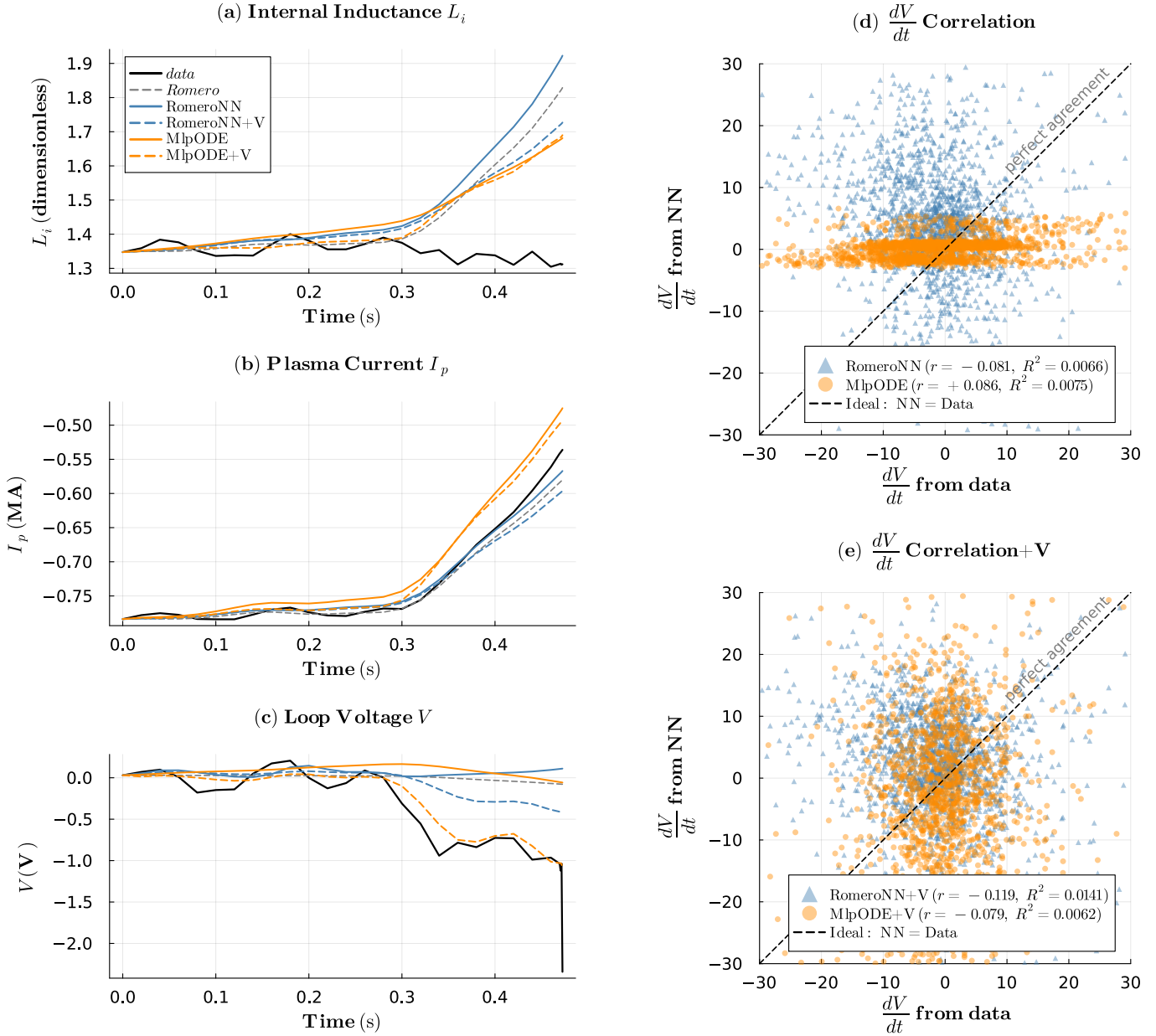


Figure 5: Julia model predictions on a held-out test discharge (shot 1150520003, the example shot from Wang et al.). (a–c) Trajectories for L_i , I_p , and V against experimental data (black) and the pure-Romero baseline (gray); RomeroNN blue, MlpODE orange, base form solid, +V dashed. The models track L_i and I_p closely while their V outputs stay smooth and low-amplitude relative to the data. (d–e) Predicted vs. finite-differenced dV/dt for the base (d) and +V (e) models, with the diagonal marking perfect agreement and Pearson r and R^2 annotated per series; the near-zero correlation — against a pure-Romero floor of $R^2 = 0.0002$ — shows the networks fit L_i and I_p without recovering V dynamics.

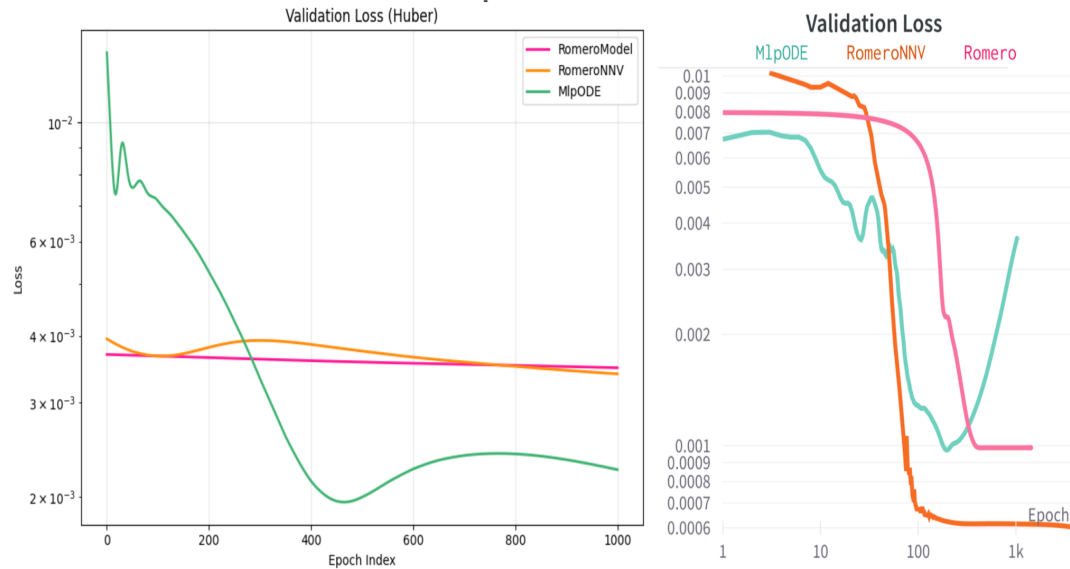


Figure 6: Validation loss curves demonstrating the overfitting susceptibility of the pure neural network (MlpODE) relative to the physics-anchored models. The left panel details the full-shot continuous evaluation from the PyTorch implementation, where the MlpODE rapidly achieves a minimal loss by treating the loop voltage as an unconstrained free variable to fit the noise-dominated data, whereas the physics-informed models (RomeroNNV and RomeroModel) reach a stable plateau. The right panel displays the corresponding loss evaluated over a 10-step multiple-shooting window directly from Wang et al., revealing that the unconstrained MlpODE eventually overfits and diverges. This contrasting behavior confirms that the pure neural network's initial rapid loss minimization is a mathematical artifact of memorizing measurement noise rather than the discovery of genuine underlying plasma physics.

Conclusion. Wang's hybrid neural ODE reproduces well and predicts L_i and I_p strongly, but equation discovery on its V channel is not viable on this dataset by standard symbolic regression or SINDy — the limiting factor is signal noise, not the modeling pipeline. The broader caution stands: a hybrid neural ODE's accuracy on its supervised channels does not entail that it has learned the physics of its unsupervised ones. This carries an ethical dimension worth stating directly. The benefit of such methods is a faster, auditable route to real-time fusion control — and thus to clean baseload energy; the risk is that a model trusted for its accuracy on the channels it can observe, yet blind to the physics it cannot, could fail in an untrained regime and provoke the very disruptions it was meant to prevent.

CRediT Authorship Contribution Statement

- **Data Curation:** Christopher Billingham (Alcator C-Mod dataset collection)
- **Methodology:** Christopher Billingham, Phillip Prior, Tino Wells
- **Software:** Christopher Billingham (Julia/SciML implementation), Phillip Prior (Python/JAX Equinox implementation), Tino Wells (Python/PyTorch implementation)
- **Investigation:** Christopher Billingham, Phillip Prior, Tino Wells (Equal contribution to the independent recreation of the hybrid Neural ODE models and cross-framework diagnostic analysis)
- **Formal Analysis:** Christopher Billingham, Phillip Prior, Tino Wells
- **Validation:** Christopher Billingham, Phillip Prior, Tino Wells
- **Writing – Original Draft:** Christopher Billingham (Introduction, Technical Background, Julia/SciML Methodology, Julia Preliminary Results); Phillip Prior (JAX Methodology, JAX Preliminary Results); Tino Wells (PyTorch Methodology, PyTorch Preliminary Results, Summary and Conclusions)
- **Writing – Review & Editing:** Christopher Billingham, Phillip Prior, Tino Wells

Citations

0. Code base: <https://github.com/caderache2014/ENGR521A/>

- [1] D. Stork and S. J. Zinkle. Introduction to the special issue on the technical status of materials for a fusion reactor. *Nuclear Fusion*. 57(9):092001, 2017 [doi: 10.1088/1741-4326/aa69e4](https://doi.org/10.1088/1741-4326/aa69e4)
- [2] Anonymous, “Deuterium-Tritium fusion reaction” diagram. Released into the public domain. Wikipedia Commons: <https://commons.wikimedia.org/wiki/File:D-t-fusion.png>
- [3] Alcator C-Mod image: M. Greenwald, et al. 20 years of research on the Alcator C-Mod tokamak. *Physics of Plasmas*, 21(11):110501, 2014 [doi: 10.1063/1.4901920](https://doi.org/10.1063/1.4901920)
- [4] T. K. Fowler, *The Fusion Quest*. The Johns Hopkins University Press, 1997.
- [5] S. E. Wurzel and S.C. Hsu. Progress toward fusion energy breakeven and gain as measured by the Lawson criterion. *Physics of Plasmas*, 29(6), 2022. [doi: 10.1063/5.0083990](https://doi.org/10.1063/5.0083990)
- [6] J. E. Menard, et al. Fusion nuclear science facilities and pilot plants based on the spherical tokamak. *Nuclear Fusion*. 56(105023), 2016 [doi: 10.1088/0029-5515/56/10/106023](https://doi.org/10.1088/0029-5515/56/10/106023)
- [7] D. Humphreys, et al. Advancing Fusion with Machine Learning Research Needs Workshop Report. *J Fusion Energy* **39**, 123–155 (2020). [doi: 10.1007/s10894-020-00258-1](https://doi.org/10.1007/s10894-020-00258-1)
- [8] T. C. Hender, et al. Chapter 3: MHD stability, operational limits and disruptions. *Nuclear Fusion*, 47(6), S128–S202. 2007 [doi: 10.1088/0029-5515/47/6/S03](https://doi.org/10.1088/0029-5515/47/6/S03)
- [9] J. Degraeve, et al. Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature*. 602(7897):414–419, 2022. [doi: 10.1038/s41586-021-04301-9](https://doi.org/10.1038/s41586-021-04301-9)
- [10] J. Kates-Harbeck, A. Svyatkovskiy, and W Tang. Predicting disruptive instabilities in controlled fusion plasmas through deep learning. *Nature* **568**, 526–531 (2019). [doi: 10.1038/s41586-019-1116-4](https://doi.org/10.1038/s41586-019-1116-4)
- [11] C. Rea, et al. Disruption prediction investigations using Machine Learning tools on DIII-D and Alcator C-Mod. *Plasma Phys. Control. Fusion*. 60:084004, 2018 [doi: 10.1088/1361-6587/aac7fe](https://doi.org/10.1088/1361-6587/aac7fe)
- [12] M. D. Boyer, C. Rea, and M. Clement. Toward active disruption avoidance via real-time estimation of the safe operating region and disruption proximity in tokamaks. *Nuclear Fusion*, 62(026005), 2021 [doi: 10.1088/1741-4326/ac359e](https://doi.org/10.1088/1741-4326/ac359e)
- [13] K. L. van de Plassche, et al. Fast modeling of turbulent transport in fusion plasmas using neural networks. *Physics of Plasmas*, 27(022310), 2020 [doi: 10.1063/1.513126](https://doi.org/10.1063/1.513126)
- [14] G. E. Karniadakis, I. G. Kevrekidis, and L. Lu. Physics-informed machine learning. *Nat Rev Phys* **3**, 422–440, 2021. [doi: 10.1038/s42254-021-00314-5](https://doi.org/10.1038/s42254-021-00314-5)
- [15] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving on-linear partial differential equations. *J. Comp. Physics*, 378:686–707, 2019 [doi: 10.1016/j.jcp.2018.10.045](https://doi.org/10.1016/j.jcp.2018.10.045)
- [16] M. Schmidt and H. Lipson. Distilling free-form natural laws from experimental data. *Science*. 324(5923):81-5, 2009 [doi: 10.1126/science.1165893](https://doi.org/10.1126/science.1165893)

- [17] M. Cranmer, *et al.* Discovering symbolic models from deep learning with inductive biases. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. [arXiv:2006.11287](https://arxiv.org/abs/2006.11287)
- [18] S. L. Brunton, J. L. Proctor, and J. N. Kutz. Discovering governing equations from data: sparse identification of dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15), 3932–3937, 2016. [doi: 10.1073/pnas.1517384113](https://doi.org/10.1073/pnas.1517384113)
- [19] A. M. Wang, D. T. Garnier, and C. Rea. Hybridizing physics and neural ODEs for predicting plasma inductance dynamics in tokamak fusion reactors. *NeurIPS 2023 Workshop on Tackling Climate Change with Machine Learning*. [arXiv:2310.20079v1](https://arxiv.org/abs/2310.20079v1)
- [20] J. Romero, JET-EFDA Contributors, *et al.* Plasma internal inductance dynamics in a tokamak. *Nuclear Fusion*, 50(100):115002, 2010 [doi: 10.1088/0029-5515/50/11/115002](https://doi.org/10.1088/0029-5515/50/11/115002)
- [21] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, pages 6572–6583, 2018. [arXiv:1806.07366](https://arxiv.org/abs/1806.07366)
- [22] A. M. Wang, private communication, December 2024
- [23] L. Lao, *et al.* Equilibrium analysis of current profiles in tokamaks. *Nuclear Fusion*, 30(6):1035, 1990.
- [24] S. Massaroli, *et al.* Differentiable multiple shooting layers. *Advances in Neural Information Processing Systems*, 34:16532-16544, 2021.
- [25] P. J. Huber, "Robust estimation of a location parameter," *Ann. Math. Statist.*, vol. 35, no. 1, pp. 73–101, 1964. [doi: 10.1214/aoms/1177703732](https://doi.org/10.1214/aoms/1177703732)
- [26] A. Pal, "Lux: Explicit parameterization of deep neural networks in Julia," *Zenodo*, Apr. 2023. [doi: 10.5281/zenodo.7808904](https://doi.org/10.5281/zenodo.7808904)
- [27] C. Rackauckas and Q. Nie, "DifferentialEquations.jl — A performant and feature-rich ecosystem for solving differential equations in Julia," *J. Open Res. Softw.*, vol. 5, no. 1, p. 15, 2017. [doi: 10.5334/jors.151](https://doi.org/10.5334/jors.151)
- [28] Y. Ma, V. Dixit, M. J. Innes, X. Guo, and C. Rackauckas, "A comparison of automatic differentiation and continuous sensitivity analysis for derivatives of differential equation solutions," in *Proc. IEEE High Performance Extreme Computing Conf. (HPEC)*, 2021, pp. 1–9. [doi: 10.1109/HPEC49654.2021.9622796](https://doi.org/10.1109/HPEC49654.2021.9622796)
- [29] V. K. Dixit and C. Rackauckas, "Optimization.jl: A unified optimization package," *Zenodo*, Mar. 2023. [doi: 10.5281/zenodo.7738525](https://doi.org/10.5281/zenodo.7738525)
- [30] D.A. Messenger and D.M. Bortz. "Weak SINDy: Galerkin-Based Data-Driven Model Selection." *Multiscale Modeling & Simulation*, edited by , vol. 19, no. 3, Jan. 2021, pp. 1474–97. [doi: 10.1137/20m1343166](https://doi.org/10.1137/20m1343166)
- [31] Fasel, U., *et al.* "Ensemble-SINDy: Robust Sparse Model Discovery in the Low-Data, High-Noise Limit, with Active Learning and Control." *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, edited by , vol. 478, no. 2260, Apr. 2022. [doi: 10.1098/rspa.2021.0904](https://doi.org/10.1098/rspa.2021.0904)
- [32] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2010.